

Shiv Nadar University Chennai

B.Tech Artificial Intelligence & Data Science

Semester V

CS3807 – Deep Learning Laboratory

Experiment 3

**Implementation of Convolutional Neural
Networks (CNNs) for Image Classification**

Name:	Atiksh Borundia
Reg. No.:	24011101009
Academic Year:	2026–27

Contents

1	Objective	2
2	Dataset	2
3	Task 1: Load and Explore CIFAR-10	2
4	Task 2: Implement a Convolution Layer	3
5	Task 3: Study the Effect of Stride and Padding	4
6	Task 4: Visualize Feature Maps	5
7	Task 5: Compare Max Pooling vs Average Pooling	6
8	Task 6: Construct and Train the CNN	6
9	Task 7: Evaluate the CNN	7
10	Mandatory Plots	9
11	Additional Exercises	10
11.1	Exercise 1: Convolution Output Size	10
11.2	Exercise 2: Trainable Parameters	10
11.3	Exercise 3: ReLU vs Sigmoid	10
11.4	Exercise 4: Average Pooling	11
11.5	Exercise 5: Effect of Number of Filters	11
12	Overall Results	13
13	Source Code	14
14	Discussion	14
14.1	Why is convolution preferred over fully connected layers for images? . . .	14
14.2	How does stride affect the feature map size?	14
14.3	What is the role of padding?	14
14.4	Why is pooling used?	14
14.5	How do feature maps represent image characteristics?	14
14.6	Why do CNNs require fewer parameters than MLPs?	15
15	Conclusion	15

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature-map visualization, and image classification using TensorFlow/Keras.

2 Dataset

CIFAR-10 contains 50,000 training images, 10,000 testing images, 10 classes, and RGB images of size $32 \times 32 \times 3$. The ten classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck.

3 Task 1: Load and Explore CIFAR-10

The training data shape is (50000, 32, 32, 3), training labels are (50000, 1), test data is (10000, 32, 32, 3), test labels are (10000, 1), and there are 10 classes.

Quantity	Result
Training images	50,000
Testing images	10,000
Training image shape	$32 \times 32 \times 3$
Testing image shape	$32 \times 32 \times 3$
Number of classes	10
Images per training class	5,000

Table 1: CIFAR-10 dataset results



Figure 1: Mandatory Plot 1: Sample Images.

Inference: The samples show that the dataset has a variety of photos with $32/32$ pixels which are not very clear but provides a good dataset for the CNN Model.

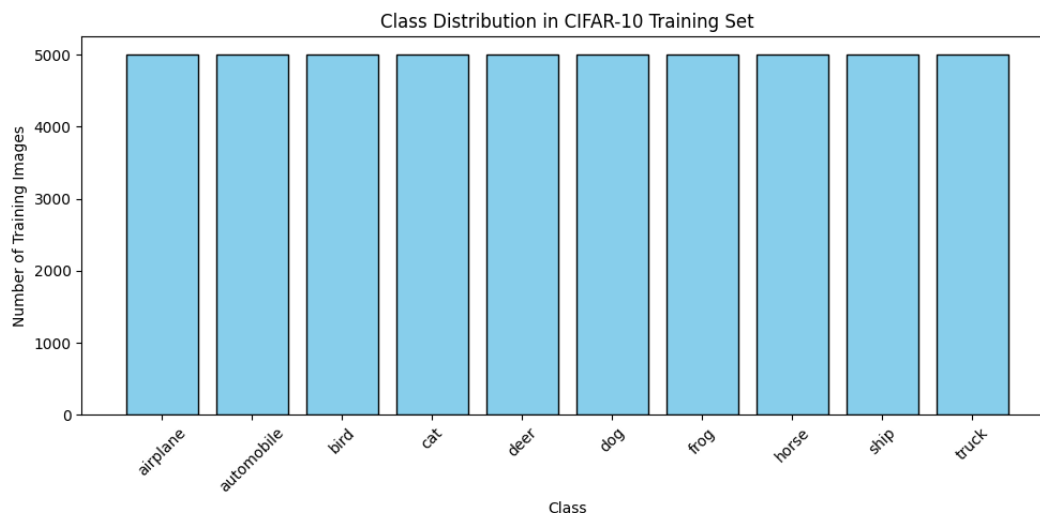


Figure 2: Mandatory Plot 2: Class Distribution.

Inference: The dataset is balanced, with 5000 samples per class. Hence we can say that accuracy may be a good measure as there may be no bias of more number of classification done towards dominant classes.

4 Task 2: Implement a Convolution Layer

A convolution layer with 8 filters was tested using kernel sizes 3×3 , 5×5 , and 7×7 , stride 1 and valid padding.

Kernel	Output shape	Spatial size
3×3	(1,30,30,8)	30×30
5×5	(1,28,28,8)	28×28
7×7	(1,26,26,8)	26×26

Table 2: Convolution output sizes

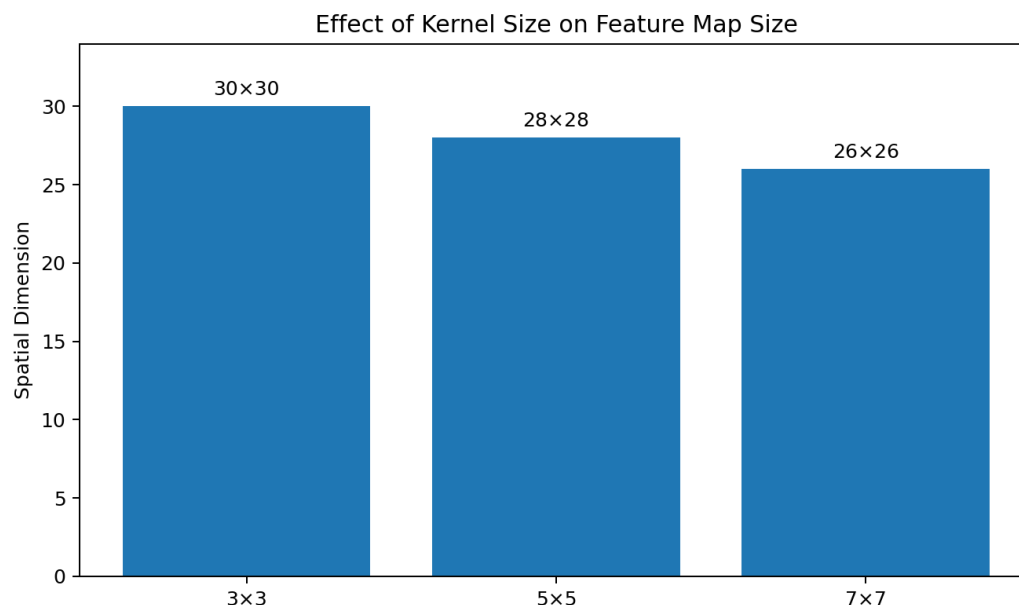


Figure 3: Kernel-size comparison.

Inference: With valid padding, no extra pixels are added around the input image, so the kernel must be inside the image. As the size increases, the kernel can be kept at fewer valid positions, so the spatial dimensions of the output feature map decrease.

5 Task 3: Study the Effect of Stride and Padding

For a 3×3 kernel the notebook compares stride and padding.

Stride	Padding	Output spatial size	Output shape
1	same	32×32	(1,32,32,8)
1	valid	30×30	(1,30,30,8)
2	same	16×16	(1,16,16,8)
2	valid	15×15	(1,15,15,8)

Table 3: Effect of stride and padding

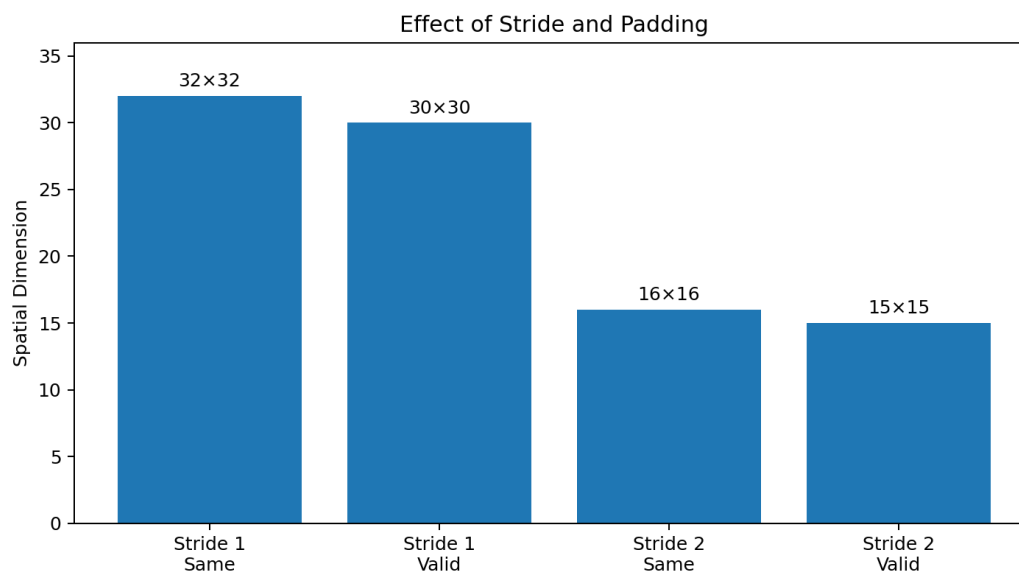


Figure 4: Stride and padding comparison.

Inference: Both stride and padding have effect on the spatial dimensions of the feature map. Increasing the stride makes the kernel move bigger steps reducing the dimensions. Valid padding reduces the output, and same padding preserves the size at stride 1 and gives ceiling otherwise.

6 Task 4: Visualize Feature Maps

A convolution layer with 16 filters was applied and feature maps were visualized.

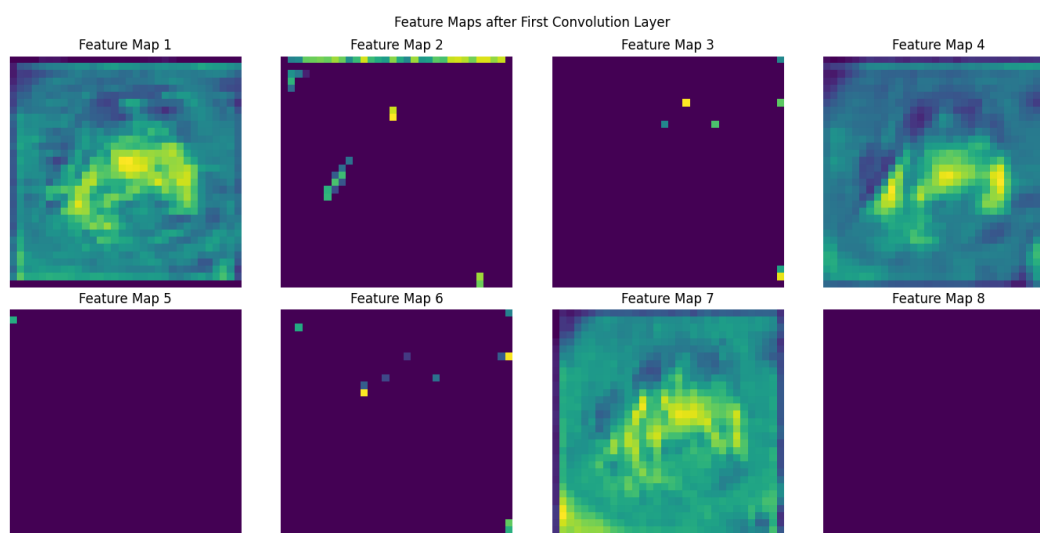


Figure 5: Mandatory Plot 7: Feature Maps.

Inference: Different filters learn different visual patterns even before the training has started. Some can respond strongly to edges, colour transitions, or textures, showing

how a CNN transforms raw pixels into learned features.

7 Task 5: Compare Max Pooling vs Average Pooling

Both pooling methods reduce the spatial dimensions from $(1, 32, 32, 16)$ to $(1, 16, 16, 16)$.

Pooling method	Final validation accuracy
Max Pooling	0.6749
Average Pooling	0.6353

Table 4: Pooling comparison

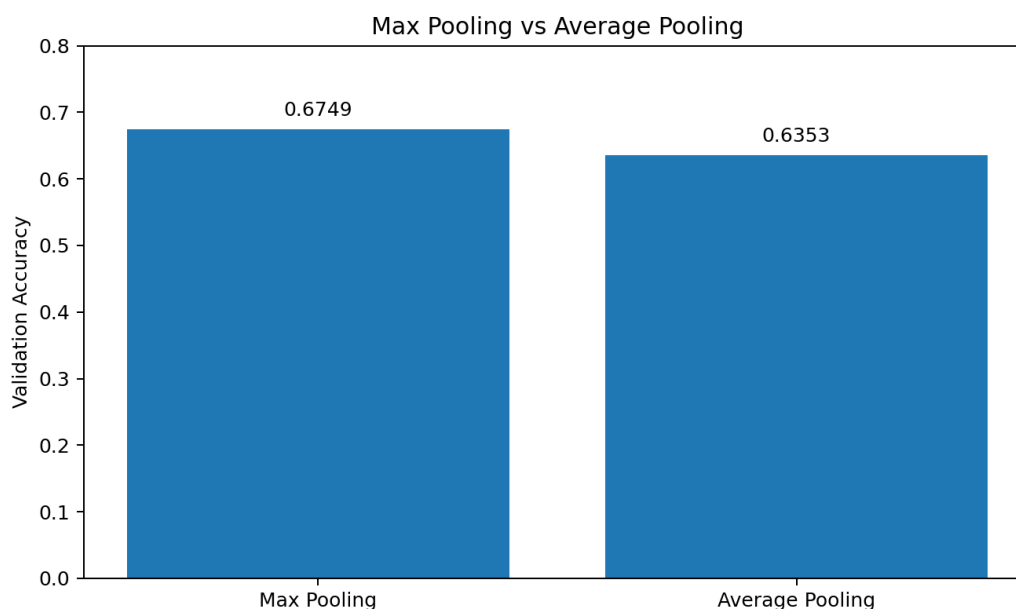


Figure 6: Max Pooling vs Average Pooling.

Inference: Max Pooling has the strongest activation in each local region, while Average Pooling smooths feature values. Here, Max Pooling gave slightly better results.

8 Task 6: Construct and Train the CNN

The architecture is Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax. The optimizer is Adam, epochs are 20, and batch size is 32.

Layer	Output shape	Parameters	Activation
Conv2D (32)	$32 \times 32 \times 32$	896	ReLU
MaxPool	$16 \times 16 \times 32$	0	–
Conv2D (64)	$16 \times 16 \times 64$	18,496	ReLU
MaxPool	$8 \times 8 \times 64$	0	–
Flatten	4096	0	–
Dense	128	524,416	ReLU
Dense	10	1,290	Softmax

Table 5: CNN architecture and parameters

Training setting	Value
Optimizer	Adam
Loss	Sparse categorical cross-entropy
Epochs	20
Batch size	32
Total parameters	545,098
Trainable parameters	545,098

Table 6: CNN training configuration

9 Task 7: Evaluate the CNN

Metric	Result
Test Accuracy	0.6657
Test Loss	2.2896
Weighted Precision	0.6779
Weighted Recall	0.6657
Weighted F1-score	0.6691

Table 7: CNN evaluation metrics

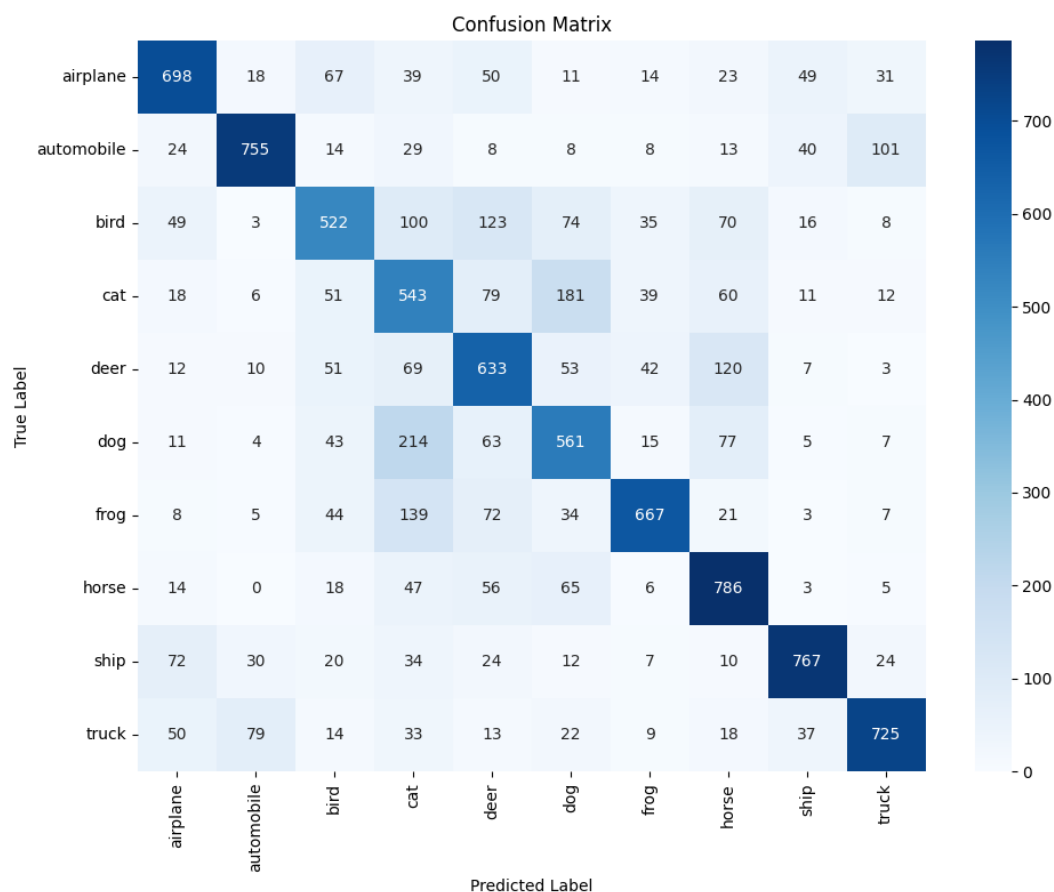


Figure 7: Mandatory Plot 8: Confusion Matrix.

Inference: The diagonal represents correct predictions. Visually similar classes have a bit of confusion as expected from the model. Clearly differentiable classes are easily classifiable. Cat and Dog are very weak precision scores here showing similarity in the animal category.

Class	Precision	Recall	F1	Support
Airplane	0.73	0.70	0.71	1000
Automobile	0.83	0.76	0.79	1000
Bird	0.62	0.52	0.57	1000
Cat	0.44	0.54	0.48	1000
Deer	0.56	0.63	0.60	1000
Dog	0.55	0.56	0.56	1000
Frog	0.79	0.67	0.72	1000
Horse	0.66	0.79	0.72	1000
Ship	0.82	0.77	0.79	1000
Truck	0.79	0.72	0.75	1000

Table 8: Classification report

10 Mandatory Plots

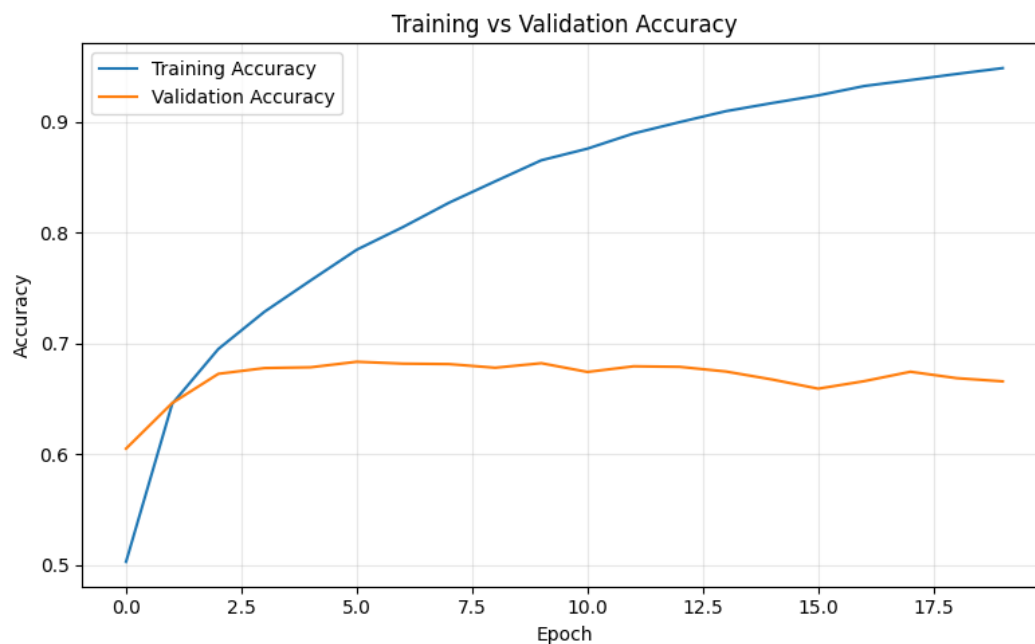


Figure 8: Mandatory Plot: Training vs Validation Accuracy.

Inference: Training accuracy has increases steadily but the stop in increase of the val. accuracy indicates that it is tending to overfit a little.



Figure 9: Mandatory Plot: Training vs Validation Loss.

Inference: Training loss decreases all the way through the experiment but the rise in validation loss confirms the earlier indication of overfitting.

11 Additional Exercises

11.1 Exercise 1: Convolution Output Size

For $N = 64$, $F = 5$, $P = 2$, and $S = 2$:

$$\frac{64 - 5 + 2(2)}{2} + 1 = 32.$$

Answer: 32×32 .

11.2 Exercise 2: Trainable Parameters

For 64 filters of size 3×3 with an RGB input:

$$(3 \times 3 \times 3 + 1) \times 64 = 1792.$$

Answer: 1792 trainable parameters.

11.3 Exercise 3: ReLU vs Sigmoid

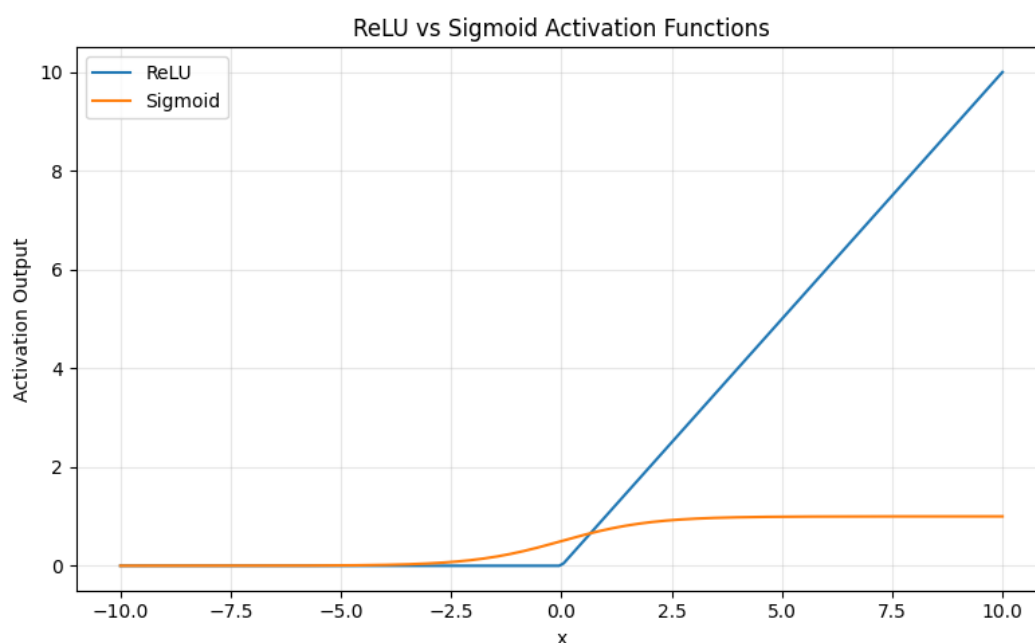


Figure 10: ReLU vs Sigmoid activation functions.

Inference: ReLU outputs zero for negative inputs and is a linear function while sigmoid can give only between 0 and 1. So, the magnitude of impact of a particular feature in CNN is reduced in sigmoid and can cause vanishing gradient problem which is widely known. Hence, RELU is preferred in CNN hidden layers and bigger models.

11.4 Exercise 4: Average Pooling

It is done in Task 5. Max Pooling obtained 0.6749 validation accuracy, while Average Pooling obtained 0.6353. which shows that MAX Pooling is slightly better than Average Pooling.

11.5 Exercise 5: Effect of Number of Filters

The filter count was increased from 16 to 64.

Filters	Validation Accuracy	Training Time
16	0.6525	178.28 s
64	0.6862	694.44 s

Table 9: Effect of number of filters

Epoch	16F Train Acc.	16F Train Loss	16F Val Acc.	16F Val Loss
1	0.4818	1.4467	0.5852	1.1754
2	0.6155	1.0952	0.6348	1.0421
3	0.6613	0.9653	0.6494	1.0002
4	0.6923	0.8794	0.6590	0.9862
5	0.7171	0.8106	0.6525	1.0222

Table 10: 16-filter experiment: all five epochs

Epoch	64F Train Acc.	64F Train Loss	64F Val Acc.	64F Val Loss
1	0.5352	1.3078	0.6155	1.0788
2	0.6749	0.9281	0.6741	0.9448
3	0.7304	0.7733	0.6911	0.9239
4	0.7727	0.6510	0.6790	0.9923
5	0.8076	0.5526	0.6862	1.0344

Table 11: 64-filter experiment: all five epochs

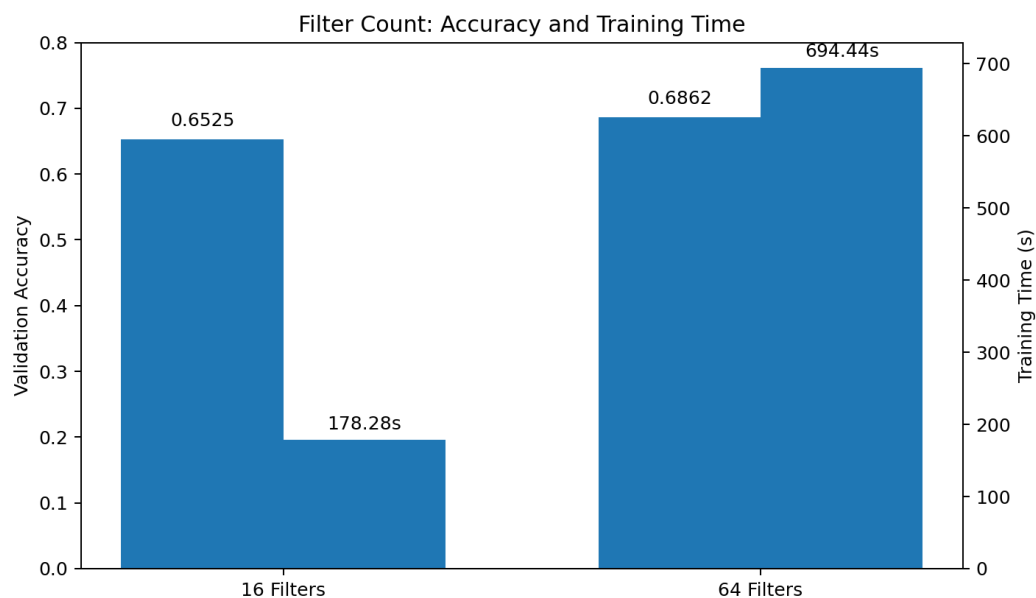


Figure 11: Filter count: validation accuracy and training time.

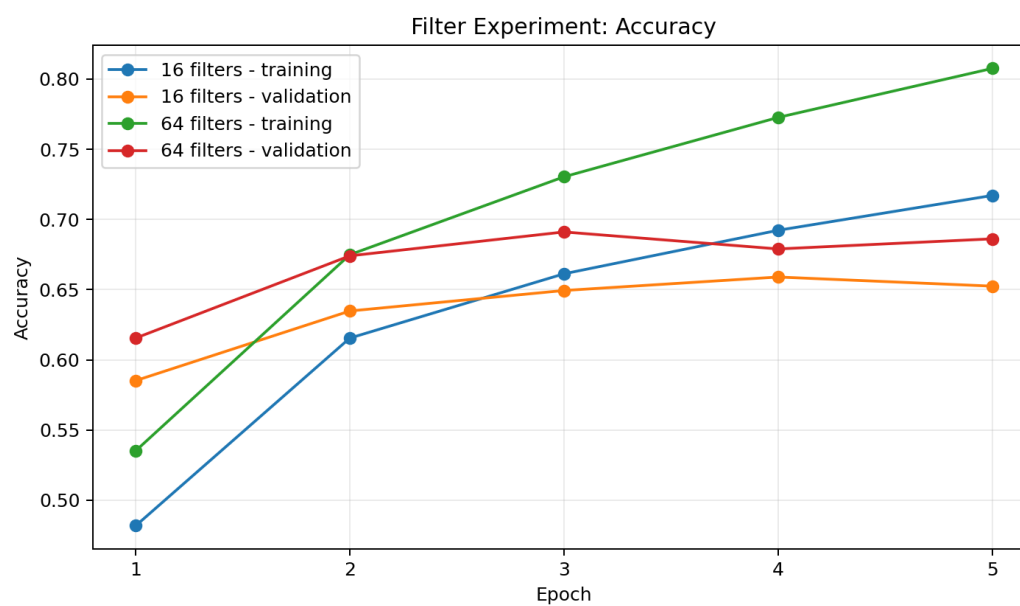


Figure 12: Filter experiment: training and validation accuracy.

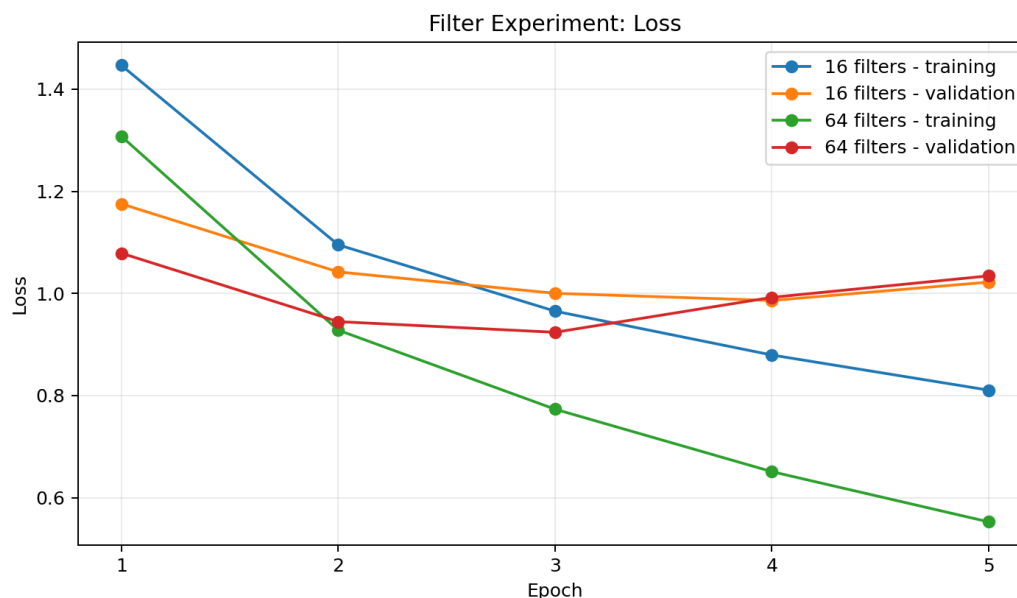


Figure 13: Filter experiment: training and validation loss.

Inference: Increasing filters from 16 to 64 has helped in increasing validation accuracy from 0.6525 to 0.6862, but training time also has increased from 178.28 s to 694.44 s. Thus, more filters provide greater representational capacity with higher computational cost. For 16 filters, validation accuracy peaked at epoch 4; for 64 filters, it peaked at epoch 3, after which validation performance became less stable showing that higher filters may lead to the peak faster epoch wise but maybe longer time wise.

12 Overall Results

Result	Value
Test Accuracy	0.6657
Test Loss	2.2896
Precision	0.6779
Recall	0.6657
F1-score	0.6691
Parameters	545,098
Max Pooling Validation Accuracy	0.6749
Average Pooling Validation Accuracy	0.6353
16 Filters Validation Accuracy	0.6525
64 Filters Validation Accuracy	0.6862
16 Filters Training Time	178.28 s
64 Filters Training Time	694.44 s

Table 12: Overall experimental results

13 Source Code

<https://colab.research.google.com/drive/1NsrE8D0nTNuHJnk9dqBqHkT0Zx0U3j0V?usp=sharing>

14 Discussion

14.1 Why is convolution preferred over fully connected layers for images?

Convolution is preferred because it shares weights and has local connectivity. Instead of having every pixel connected to each neuron, weights are reused with help of filters at various positions. In contrast, a fully connected layer operating directly on a $32 \times 32 \times 3$ image would require so many number of parameters. It also preserves the spatial structure of the image, and it helps in the same feature to get detected even when it occurs at different locations.

14.2 How does stride affect the feature map size?

Larger stride means smaller output with the results showing this way. It also controls how much spatial detail is removed, sometimes acting as downsampling.

14.3 What is the role of padding?

Padding controls the shrinking of the output, Valid loses border information and shrinks the each layer while same adds 0s

Therefore, padding is useful for controlling how quickly the spatial dimensions decrease as convolutional layers are added.

14.4 Why is pooling used?

Pooling is used to reduce the spatial dimensions of feature maps while retaining important information.

14.5 How do feature maps represent image characteristics?

A feature map represents the response produced by one convolutional filter at different locations in an image. A filter can learn to respond strongly to particular visual patterns such as edges, colour transitions, textures, or other local structures.

14.6 Why do CNNs require fewer parameters than MLPs?

CNNs require fewer parameters mainly because of **weight sharing** and **local connectivity**. A convolutional filter uses the same set of weights at every spatial location instead of learning a separate set of weights for every pixel.

15 Conclusion

So to conclude, The experiment demonstrated the main use of CNNs using CIFAR-10. Convolution, stride, padding, pooling and feature maps were used before constructing and evaluating the CNN. The final model achieved 0.6657 test accuracy, 0.6779 weighted precision, 0.6657 weighted recall and 0.6691 weighted F1-score. The filter experiment showed that increasing filters improved validation accuracy but substantially increased training time.